



Digital Lottery Platform

จัดทำโดย

นายจันทร์พงศ์	วิทย์อรุณธานี	6609650210
นางสาวนันทชำนัน	ชาคริตบุษบง	6609650467
นางสาวมาริษา	จันทรทอง	6609650616
นายเสฏฐวุฒิ	วิจิตรศิลป์	6609650723

เสนอ

อาจารย์กฤษฎิญา บุญธีรนนท์

โครงการนี้เป็นส่วนหนึ่งของวิชา คพ.362 การออกแบบซอฟต์แวร์และระบบ
ภาควิชาวิทยาการคอมพิวเตอร์ มหาวิทยาลัยธรรมศาสตร์ (ศูนย์รังสิต)
ภาคการศึกษาที่ 2 ปีการศึกษา 2568

Digital Lottery Platform — System Requirements

1. Functional Requirements (FR)

ID	Feature	คำอธิบาย
FR-01	สมัครสมาชิก	ผู้ใช้สมัครบัญชีด้วยอีเมล+เบอร์โทร+ผูกบัญชีธนาคาร
FR-02	เข้าสู่ระบบ	Login ด้วย Email/Password ระบบออก Token ให้ใช้งาน
FR-03	ค้นหาและเลือกสลาก	ค้นหาหมายเลขสลากที่ต้องการจากคลังสลาก
FR-04	จองสลาก	ล็อกสลากชั่วคราว 5 นาทีเพื่อป้องกันการซื้อซ้ำ
FR-05	ชำระเงิน	ชำระผ่าน PromptPay/บัตรเครดิต
FR-06	ดูคลังสลาก	ผู้ใช้ดูสลากที่ตัวเองซื้อแล้วทั้งหมด
FR-07	ประกาศผลรางวัล	Admin บันทึกผลออกรางวัล ระบบเผยแพร่สู่ผู้ใช้
FR-08	ตรวจผลรางวัลอัตโนมัติ	ระบบเปรียบเทียบสลากของผู้ใช้กับผลรางวัลอัตโนมัติ
FR-09	รับเงินรางวัล	โอนเงินรางวัลเข้าบัญชีธนาคารที่ผูกไว้
FR-10	แจ้งเตือน	แจ้งผลการซื้อและผลรางวัลผ่าน Push Notification
FR-11	คูประวัติรายการ	คูประวัติการซื้อและรายการย้อนหลังได้
FR-12	Audit Log	ทุก Action สำคัญถูกบันทึกเพื่อตรวจสอบความโปร่งใส

2. Non-Functional Requirements (NFR)

2.1 Scalability (การรองรับผู้ใช้จำนวนมาก)

ID	Requirement	รายละเอียด
NFR-S01	Horizontal Scaling	เพิ่ม Server ได้โดยไม่ต้องหยุดระบบ เมื่อมีผู้ใช้เพิ่มขึ้น
NFR-S02	Load Balancing	กระจาย Request ไปยัง Server หลายตัวเพื่อไม่ให้ตัวใดตัวหนึ่งรับภาระเกิน
NFR-S03	Peak Traffic	รองรับผู้ใช้พร้อมกัน $\geq 50,000$ คน ในช่วงปิดรับสลาก

2.2 Data Strategy (การจัดการข้อมูล)

ID	Requirement	รายละเอียด
NFR-D01	SQL (PostgreSQL)	เก็บข้อมูลสำคัญ เช่น ผู้ใช้, คำสั่งซื้อ, การชำระเงิน, รางวัล ที่ต้องการความถูกต้อง 100%
NFR-D02	NoSQL (MongoDB)	เก็บข้อมูลสลากจำนวนมากที่ต้องค้นหาได้เร็วและ schema ยืดหยุ่น
NFR-D03	Redis Cache	เก็บข้อมูลชั่วคราว เช่น การจองสลาก, Session Token เพื่อลดภาระ Database

2.3 Communication (การสื่อสารระหว่าง Services)

ID	Requirement	รายละเอียด
NFR-C01	REST API	ใช้สำหรับการสื่อสารทั่วไประหว่าง Client กับ Server
NFR-C02	Message Queue (RabbitMQ)	ใช้สำหรับงานที่ไม่ต้องรอผล เช่น ส่งแจ้งเตือน, ตรวจสอบรางวัล ช่วยรับมือ traffic สูงได้
NFR-C03	WebSocket	ใช้สำหรับแสดงผลรางวัล Real-time บนหน้าเว็บ

2.4 Resiliency (การรับมือเมื่อระบบมีปัญหา)

ID	Requirement	รายละเอียด
NFR-R01	Timeout & Retry	ถ้า Payment Gateway ไม่ตอบภายใน 10 วินาที ให้ลองใหม่อัตโนมัติ
NFR-R02	Reservation Timeout	สลากที่จองแต่ไม่ชำระใน 5 นาที จะถูกปล่อยคืนอัตโนมัติ
NFR-R03	Database Replication	มี Database สำรอง ถ้าตัวหลักล่ม ระบบยังทำงานได้
NFR-R04	Queue Buffer	ถ้า Notification Service ล่มชั่วคราว Message ยังไม่หาย เพราะรออยู่ใน Queue

2.5 Security & Transparency

ID	Requirement	รายละเอียด
NFR-SEC01	HTTPS	ข้อมูลทุกอย่างเข้ารหัสระหว่างส่ง
NFR-SEC02	Token-based Auth	ทุก Request ต้องมี Token ที่ออกตอน Login
NFR-SEC03	Audit Log	ทุก Action ที่สำคัญ (ซื้อ, ชำระ, รับรางวัล) ถูกบันทึกย้อนหลังได้
NFR-SEC04	Result Immutability	ผลรางวัลที่ประกาศแล้วแก้ไขไม่ได้ มี Log กำกับทุกรอบ

3. System Scope

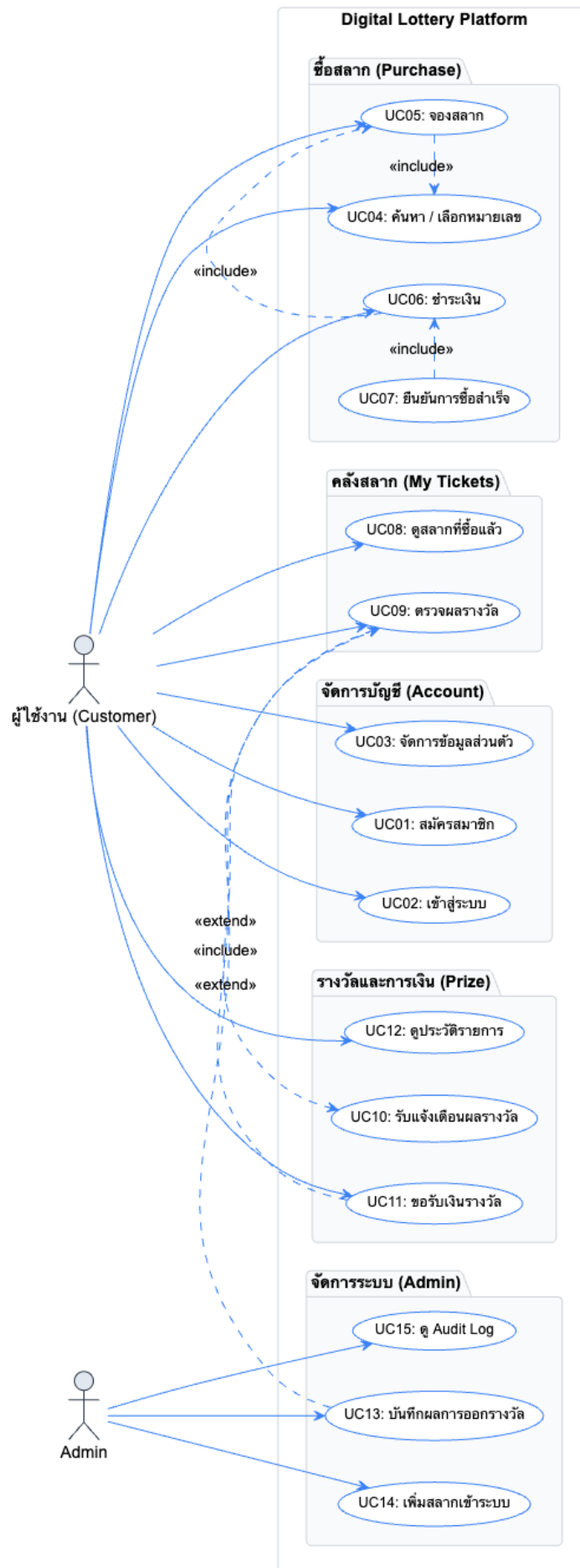
ครอบคลุม	ไม่ครอบคลุม
การจำหน่ายสลากออนไลน์	การพิมพ์สลากจริง
การประมวลผลการออกรางวัล	การจำหน่ายแบบออฟไลน์
การเผยแพร่ผลรางวัล	-
การตรวจสอบความถูกต้อง	-

4. Technology Stack

Layer	Technology	เหตุผล
Frontend	React.js	รองรับ Real-time update ผ่าน WebSocket
Load Balancer	Nginx	กระจาย Traffic ไปหลาย Server
Backend	Node.js + Express	รองรับ Request จำนวนมากพร้อมกัน
Database หลัก	PostgreSQL (SQL)	ความถูกต้องของข้อมูลทางการเงิน
Database สลาก	MongoDB (NoSQL)	ค้นหาสลากเร็ว, schema ยืดหยุ่น
Cache & Lock	Redis	ลดการจอง, Session, ลดภาระ DB
Message Queue	RabbitMQ	รับมือ traffic สูง, Async notification

Notification	Firebase FCM	Push Notification
Payment	PromptPay / Stripe	มาตรฐานการชำระเงิน

Digital Lottery Platform — Use Case Diagram



1. ผู้มีส่วนได้ส่วนเสีย (Actors)

- **ผู้ใช้งาน (Customer):** ผู้ใช้งานทั่วไปที่เข้ามาใช้บริการซื้อสลากและตรวจสอบรางวัล
- **ผู้ดูแลระบบ (Admin):** ผู้ดูแลที่มีสิทธิ์ในการจัดการข้อมูลสลากและตรวจสอบบันทึกการทำงานของระบบ (Audit Log)

2. การจัดโครงสร้างระบบ (System Modularization)

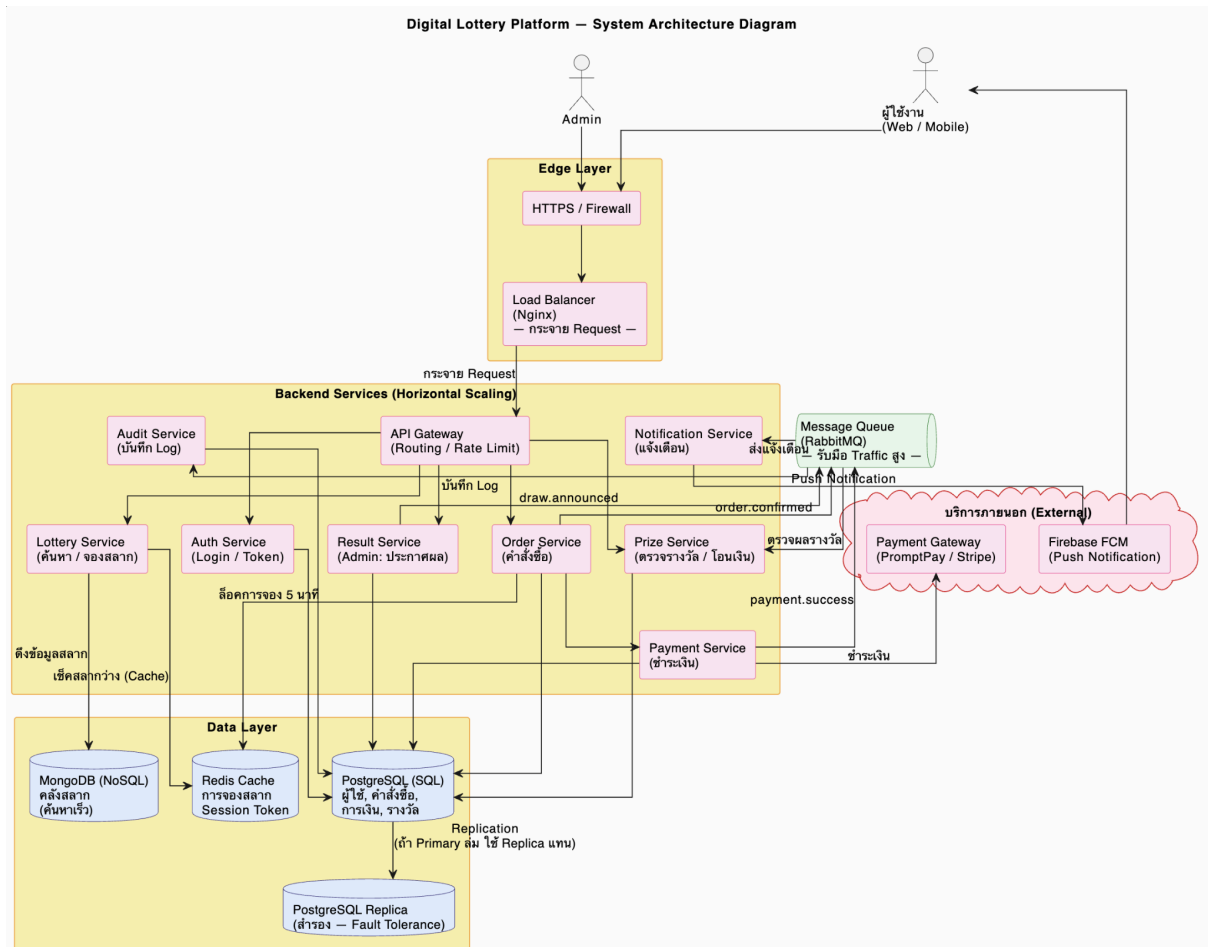
เพื่อให้ระบบสามารถดูแลรักษาและต่อยอดได้ง่าย (Scalability & Maintainability) จึงได้มีการแบ่งการทำงานออกเป็น 5 โมดูลหลัก ดังนี้

- **Account:** จัดการข้อมูลประจำตัวผู้ใช้งาน
- **Purchase:** กระบวนการซื้อสลากตั้งแต่การเลือกจนถึงการชำระเงิน
- **My Tickets:** การจัดการคลังสลากที่ครอบครอง
- **Prize:** การจัดการเรื่องเงินรางวัลและประวัติการเงิน
- **Admin:** ส่วนควบคุมหลักของระบบ

3. ความสัมพันธ์และตรรกะการทำงาน (Relationships & Logic)

การออกแบบความสัมพันธ์ในระบบลอตเตอรี่ดิจิทัลนี้ถูกกำหนดให้สะท้อนขั้นตอนการทำงานจริงอย่างเคร่งครัด โดยความสัมพันธ์แบบ <<include>> ใช้เพื่อกำหนดขั้นตอนที่เป็นข้อกำหนดบังคับในทุกธุรกรรม ได้แก่ การจองสลาก (UC05) ต้องรวมขั้นตอนการเลือกหมายเลข (UC04) เพื่อระบุสิทธิ์ก่อนดำเนินการ, การชำระเงิน (UC06) ต้องรวมการจองสลาก (UC05) เพื่อรับประกันสถานะการถือครองสิทธิ์ในระหว่างทำรายการ, การยืนยันการซื้อสำเร็จ (UC07) ต้องรวมการชำระเงิน (UC06) เพื่อเป็นจุดสิ้นสุดของธุรกรรมที่สมบูรณ์และการขอรับเงินรางวัล (UC11) ต้องรวมการตรวจผลรางวัล (UC09) เพื่อตรวจสอบสิทธิ์รับรางวัลก่อนการเบิกจ่ายเสมอ ส่วนความสัมพันธ์แบบ <<extend>> ถูกนำมาใช้สำหรับฟังก์ชันที่ทำงานภายใต้เงื่อนไขเฉพาะ ได้แก่ การตรวจผลรางวัล (UC09) ขยายการทำงานด้วยการแจ้งเตือน (UC10) เพื่อเพิ่มความสะดวกให้ผู้ใช้ในกรณีที่ต้องการทราบผลโดยไม่ต้องเข้าตรวจสอบเอง และการบันทึกผลการออกรางวัล (UC13) ขยายขอบเขตของการตรวจผลรางวัล (UC09) เพื่อให้ข้อมูลผลรางวัลอัปเดตสถานะการมองเห็นทันทีหลังจากระบบจัดการข้อมูล

Digital Lottery Platform — System Architecture Diagram



1. ภาพรวมสถาปัตยกรรม (High-Level Overview)

ระบบ Digital Lottery Platform ออกแบบในรูปแบบ **Microservices Architecture** โดยแบ่งโครงสร้างออกเป็น 5 ชั้นหลัก ได้แก่ External Services, Edge Layer, Backend Services, Message Queue และ Data Layer เพื่อรองรับการทำงานที่มีผู้ใช้งานจำนวนมากพร้อมกัน โดยเฉพาะในช่วงปิดรับสลาก

2. รายละเอียดแต่ละชั้น (Layer Description)

ชั้นที่ 1 Edge Layer

เป็นจุดแรกที่รับ Request จากผู้ใช้งาน ประกอบด้วย 2 ส่วนหลัก

HTTPS / Firewall ทำหน้าที่เข้ารหัสข้อมูลทุก Request ด้วย HTTPS และกรอง Traffic ที่ไม่ปลอดภัยออก เช่น การโจมตีแบบ SQL Injection และ XSS

Load Balancer (Nginx) ทำหน้าที่กระจาย Request ที่เข้ามาไปยัง Backend Server หลายตัวอย่างสม่ำเสมอ รองรับการทำ **Horizontal Scaling** คือสามารถเพิ่ม Server ได้เรื่อยๆ โดยไม่ต้องหยุดระบบ รองรับผู้ใช้งานพร้อมกันได้มากกว่า 50,000 คน

ชั้นที่ 2 Backend Services (Microservices)

เป็นส่วนประมวลผลหลักของระบบ แบ่งออกเป็น Service ย่อยตาม ความรับผิดชอบ ดังนี้

Service	หน้าที่
API Gateway	รับ Request ทั้งหมดจาก Client แล้ว Route ไปยัง Service ที่เกี่ยวข้อง พร้อมทำ Rate Limiting เพื่อป้องกัน Request ล้นระบบ
Auth Service	จัดการการ Login และออก Token สำหรับยืนยันตัวตนในทุก Request
Lottery Service	รับผิดชอบการค้นหาและแสดงข้อมูลสลากจากคลัง โดยดึงข้อมูลจาก MongoDB และตรวจสอบสถานะผ่าน Redis Cache ก่อนเสมอ
Order Service	จัดการคำสั่งซื้อ ตั้งแต่การจองสลากด้วย Redis Lock ไปจนถึงการสร้าง Order
Payment Service	ติดต่อกับ Payment Gateway ภายนอก (PromptPay / Stripe) เพื่อดำเนินการชำระเงิน พร้อม Retry อัตโนมัติหาก Gateway ไม่ตอบสนอง
Prize Service	ตรวจสอบสลากของผู้ใช้กับผลรางวัลโดยอัตโนมัติหลังประกาศผล และจัดการการโอนเงินรางวัล
Result Service	รองรับการบันทึกผลรางวัลโดย Admin และเผยแพร่ไปยัง Services อื่นผ่าน Message Queue
Notification Service	รับ Event จาก Message Queue แล้วส่ง Push Notification ไปยังผู้ใช้ผ่าน Firebase FCM

Audit Service	บันทึกทุก Action สำคัญในระบบ เพื่อให้สามารถตรวจสอบย้อนหลังได้ตลอดเวลา
----------------------	---

ชั้นที่ 3 Message Queue (RabbitMQ)

ระบบใช้ **RabbitMQ** เป็น Message Queue สำหรับงานที่ไม่จำเป็นต้องรอผลทันที (Asynchronous) เช่น การส่งแจ้งเตือน การตรวจผลรางวัล และการบันทึก Log

การออกแบบนี้มีประโยชน์ 2 ด้านหลัก ได้แก่ ในด้าน **Scalability** ช่วยรับมือ Traffic สูงในช่วงปิดรับสลาก เพราะ Service ไม่ต้องประมวลผลทุกอย่างพร้อมกัน และในด้าน **Resiliency** หาก Notification Service ล่มชั่วคราว Message จะยังคงรออยู่ใน Queue และถูกส่งอีกครั้งเมื่อ Service กลับมาทำงาน โดยข้อมูลไม่สูญหาย

ชั้นที่ 4 Data Layer

ระบบเลือกใช้ Database หลายประเภทตามลักษณะของข้อมูล

PostgreSQL (SQL) ใช้เก็บข้อมูลสำคัญที่ต้องการความถูกต้อง 100% เช่น ข้อมูลผู้ใช้ คำสั่งซื้อ การชำระเงิน และรางวัล เนื่องจาก PostgreSQL รองรับ Transaction แบบ ACID ซึ่งมีความสำคัญมากสำหรับข้อมูลทางการเงิน

PostgreSQL Replica คือ Database สำรองที่รับข้อมูลต่อจาก Primary แบบ Real-time หากเซิร์ฟเวอร์หลักล่ม ระบบจะสลับมาใช้ Replica โดยอัตโนมัติ

MongoDB (NoSQL) ใช้เก็บข้อมูลคลังสลากจำนวนมาก เนื่องจากโครงสร้างข้อมูลสลากมีความยืดหยุ่นและต้องการความเร็วในการค้นหาสูง MongoDB จึงเหมาะสมกว่า SQL ในกรณีนี้

Redis Cache ใช้เก็บข้อมูลชั่วคราวในหน่วยความจำ มีบทบาทสำคัญ 2 อย่าง ได้แก่ การ Cache สถานะสลากเพื่อลดภาระ Database และการทำ Distributed Lock เพื่อป้องกันผู้ใช้หลายคนจองสลากใบเดียวกันพร้อมกัน

ชั้นที่ 5 External Services

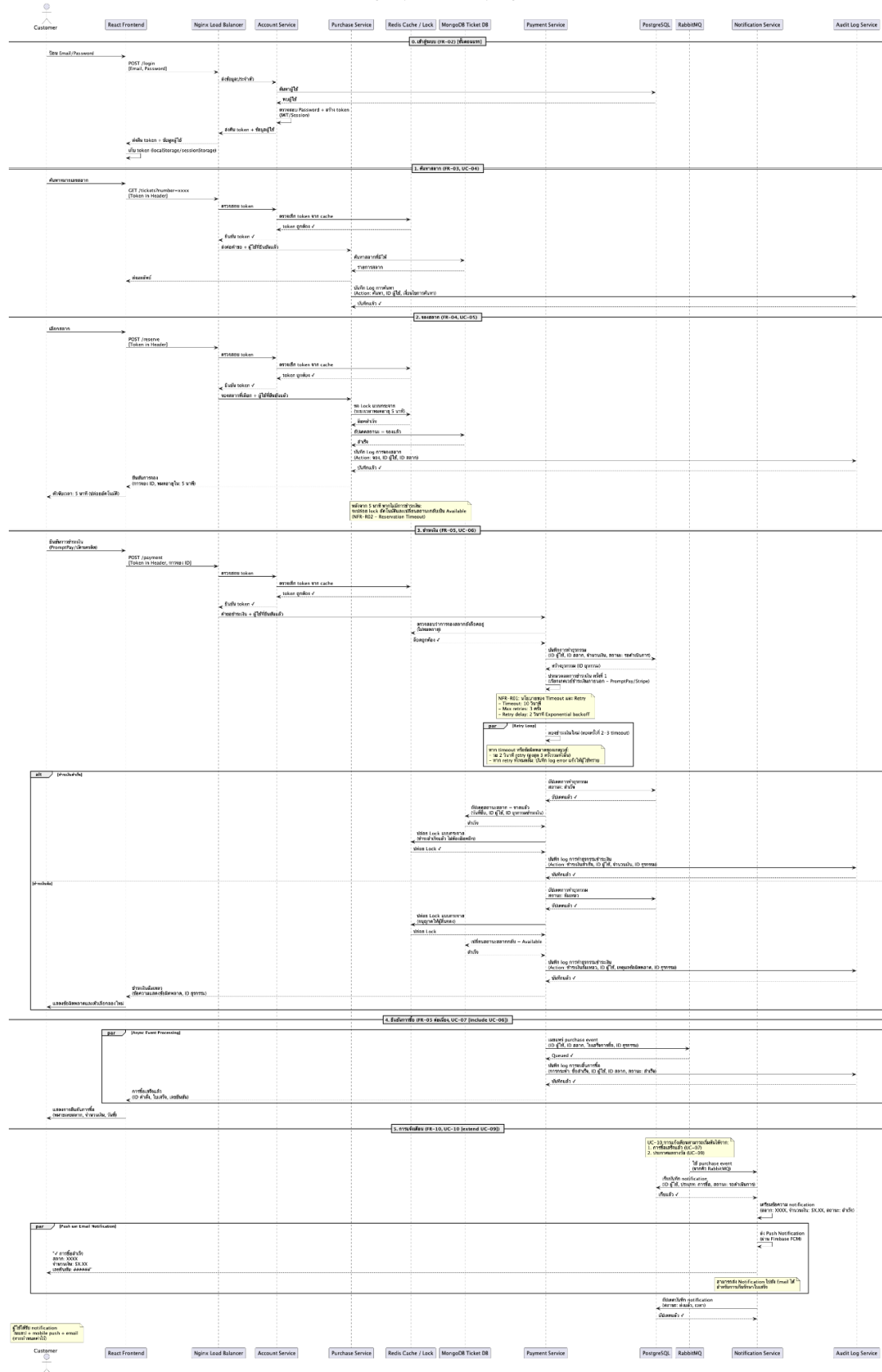
ระบบเชื่อมต่อกับบริการภายนอก 2 ส่วน ได้แก่ **Payment Gateway (PromptPay / Stripe)** สำหรับดำเนินการชำระเงิน และ **Firebase FCM** สำหรับส่ง Push Notification ไปยังอุปกรณ์ของผู้ใช้

3. Communication Protocol

ระบบเลือกใช้โปรโตคอลการสื่อสารที่เหมาะสมกับแต่ละกรณี ดังนี้

โปรโตคอล	การใช้งาน	ตัวอย่าง
REST API	การสื่อสารทั่วไประหว่าง Client กับ Server	ค้นหาสลาก, สร้าง Order, ชำระเงิน
Message Queue (RabbitMQ)	งาน Async ที่ไม่ต้องรอผลทันที	ส่งแจ้งเตือน, ตรวจสอบรางวัล, บันทึก Log
WebSocket	การแสดงผลแบบ Real-time	แสดงผลรางวัลสดบนหน้าเว็บ

Digital Lottery Platform – Purchase Ticket Sequence Diagram



1. ภาพรวม Sequence Diagram

Sequence Diagram แสดงลำดับการทำงานและการสื่อสารระหว่าง Components ต่างๆของระบบ ตั้งแต่ขั้นตอนผู้ใช้เข้าสู่ระบบ ไปจนถึงการรับแจ้งเตือน โดยครอบคลุม กระบวนการหลักที่เชื่อมโยงกันอย่างเป็นระบบ

ระบบประกอบด้วย Components ดังต่อไปนี้

Component	เทคโนโลยี	บทบาทหน้าที่
React Frontend	React.js	ส่วนติดต่อผู้ใช้ รับ Input และแสดงผลลัพธ์
Nginx Load Balancer	Nginx	กระจาย Request ไปยัง Service ต่าง ๆ และเป็น Entry Point
Account Service	Node.js	จัดการ Authentication และตรวจสอบ Token ทุก Request
Perchase Service	Node.js	ควบคุมกระบวนการค้นหาและจองสลาก
Redis Cache / Lock	Redis	เก็บ Session Token และจัดการ Distributed Lock การจองสลาก
MongoDB Ticket DB	MongoDB	ฐานข้อมูลสลาก รองรับการค้นหาเร็วและ Schema ยืดหยุ่น
Payment Service	Node.js	ประมวลผลการชำระเงิน เชื่อมต่อ PromptPay และ Stripe
PostgreSQL	PostgreSQL	ฐานข้อมูลหลัก เก็บข้อมูลผู้ใช้ คำสั่งซื้อ และธุรกรรมการเงิน
RabbitMQ	RabbitMQ	Message Queue สำหรับ Async Processing เช่น การแจ้งเตือน
Notification Service	Firebase FCM	ส่ง Push Notification และ Email แจ้งผลการซื้อสลาก
Audit Log Service	Node.js	บันทึกทุก Action สำคัญเพื่อความโปร่งใสและการตรวจสอบย้อนหลัง

2. ขั้นตอนการทำงาน

กระบวนการทำงาน แบ่งออกเป็น 6 ขั้นตอนหลักที่ดำเนินการตามลำดับ

ขั้นตอน	ชื่อกระบวนการ	คำอธิบาย
0	เข้าสู่ระบบ	ผู้ใช้อยืนยันตัวตนด้วย Email/Password ระบบออก JWT Token สำหรับใช้ใน Request ถัดไป
1	ค้นหาสลาก	ผู้ใช้นำหมายเลขสลากที่ต้องการ ระบบดึงข้อมูลจาก MongoDB และบันทึก Log
2	จองสลาก	ระบบ Lock สลากที่เลือกด้วย Distributed Lock ใน Redis นาน 5 นาที เพื่อป้องกันการซื้อซ้ำ
3	ชำระเงิน	ชำระผ่าน PromptPay หรือบัตรเครดิต มีระบบ Retry อัตโนมัติสูงสุด 3 ครั้ง หาก Gateway ไม่ตอบสนอง
4	ยืนยันการซื้อ	ระบบออกใบเสร็จและเลขยืนยัน พร้อมเผยแพร่ Event เข้า RabbitMQ แบบ Async
5	แจ้งเตือน	Notification Service รับ Event จาก Queue แล้วส่ง Push Notification ผ่าน Firebase FCM

3. รายละเอียดแต่ละขั้นตอน

ขั้นตอนที่ 0 — เข้าสู่ระบบ (FR-02)

เมื่อผู้ใช้อกรอก Email และ Password ที่ React Frontend แล้วกดเข้าสู่ระบบ Frontend จะส่ง POST /login ผ่าน Nginx Load Balancer ไปยัง Account Service Account Service จะค้นหาข้อมูลผู้ใช้จาก PostgreSQL จากนั้นตรวจสอบ Password และสร้าง JWT Token เมื่อยืนยันตัวตนสำเร็จ Token พร้อมข้อมูลผู้ใช้จะถูกส่งกลับผ่าน Load Balancer มายัง Frontend เพื่อเก็บไว้ใน Session Storage สำหรับการใช้งานในขั้นตอนถัดไป

NFR-SEC02: ทุก Request หลังจากนี้จะต้องแนบ Token ใน HTTP Header เพื่อยืนยันสิทธิ์การใช้งาน

ขั้นตอนที่ 1 — ค้นหาและเลือกสลาก (FR-03, UC-04)

ผู้ใช้ระบุหมายเลขสลากที่ต้องการค้นหาผ่าน Frontend ซึ่งจะส่ง GET /tickets?number=xxxx พร้อม Token ใน Header ทุก Request ที่ผ่าน Load Balancer จะถูกตรวจสอบ Token กับ Account Service โดย Account Service จะตรวจเช็คความถูกต้องของ Token จาก Redis Cache เพื่อลดภาระการเข้าถึงฐานข้อมูลหลัก เมื่อยืนยัน Token สำเร็จ Purchase Service จะค้นหาข้อมูลสลากที่มีสถานะ Available จาก MongoDB และส่งผลลัพธ์กลับสู่ Frontend พร้อมบันทึก Audit Log

ขั้นตอนที่ 2 — จองสลาก (FR-04, UC-05)

เมื่อผู้ใช้เลือกสลากที่ต้องการ Purchase Service จะร้องขอ Distributed Lock จาก Redis โดยกำหนดเวลาหมดอายุ 5 นาที หาก Lock สำเร็จ ระบบจะอัปเดตสถานะสลากใน MongoDB เป็น "จองแล้ว" และแจ้งให้ผู้ใช้ทราบพร้อมนับถอยหลัง 5 นาที ระหว่างนี้ผู้ใช้รายอื่นจะไม่สามารถจองสลากหมายเลขเดียวกันได้ หากผู้ใช้ไม่ดำเนินการชำระเงินภายในเวลาที่กำหนด Redis จะปล่อย Lock อัตโนมัติ และระบบจะเปลี่ยนสถานะสลากกลับเป็น Available เพื่อให้ผู้ใช้รายอื่นสามารถจองได้

NFR-R02: Reservation Timeout — สลากที่จองแต่ไม่ชำระภายใน 5 นาที จะถูกปล่อยคืนสู่ระบบโดยอัตโนมัติ

ขั้นตอนที่ 3 — ชำระเงิน (FR-05, UC-06)

ผู้ใช้อยืนยันการชำระเงินผ่านช่องทางที่เลือก ได้แก่ PromptPay หรือบัตรเครดิต Payment Service จะตรวจสอบว่า Reservation Lock ยังคงอยู่ใน Redis ก่อนดำเนินการ จากนั้นบันทึกธุรกรรมใน PostgreSQL พร้อมสถานะ "รอดำเนินการ" แล้วเรียก External Payment Gateway โดยมีนโยบาย Timeout 10 วินาทีต่อครั้ง และ Retry อัตโนมัติสูงสุด 3 ครั้ง ด้วย Exponential Backoff

ผลลัพธ์การชำระเงินแบ่งเป็น 2 กรณี ได้แก่

- ชำระสำเร็จ: อัปเดตสถานะธุรกรรมใน PostgreSQL เป็น "สำเร็จ" อัปเดตสถานะสลากใน MongoDB เป็น "ขายแล้ว" ปล่อย Distributed Lock และบันทึก Audit Log
- ชำระล้มเหลว: อัปเดตสถานะธุรกรรมเป็น "ล้มเหลว" ปล่อย Lock คืน เปลี่ยนสถานะสลากกลับเป็น Available และแจ้งผู้ใช้เพื่อลองใหม่

NFR-R01: Timeout & Retry Policy — Payment Gateway Timeout 10 วินาที, Retry สูงสุด 3 ครั้ง พร้อม Exponential Backoff 2 วินาที

ขั้นตอนที่ 4 — ยืนยันการซื้อสำเร็จ (UC-07)

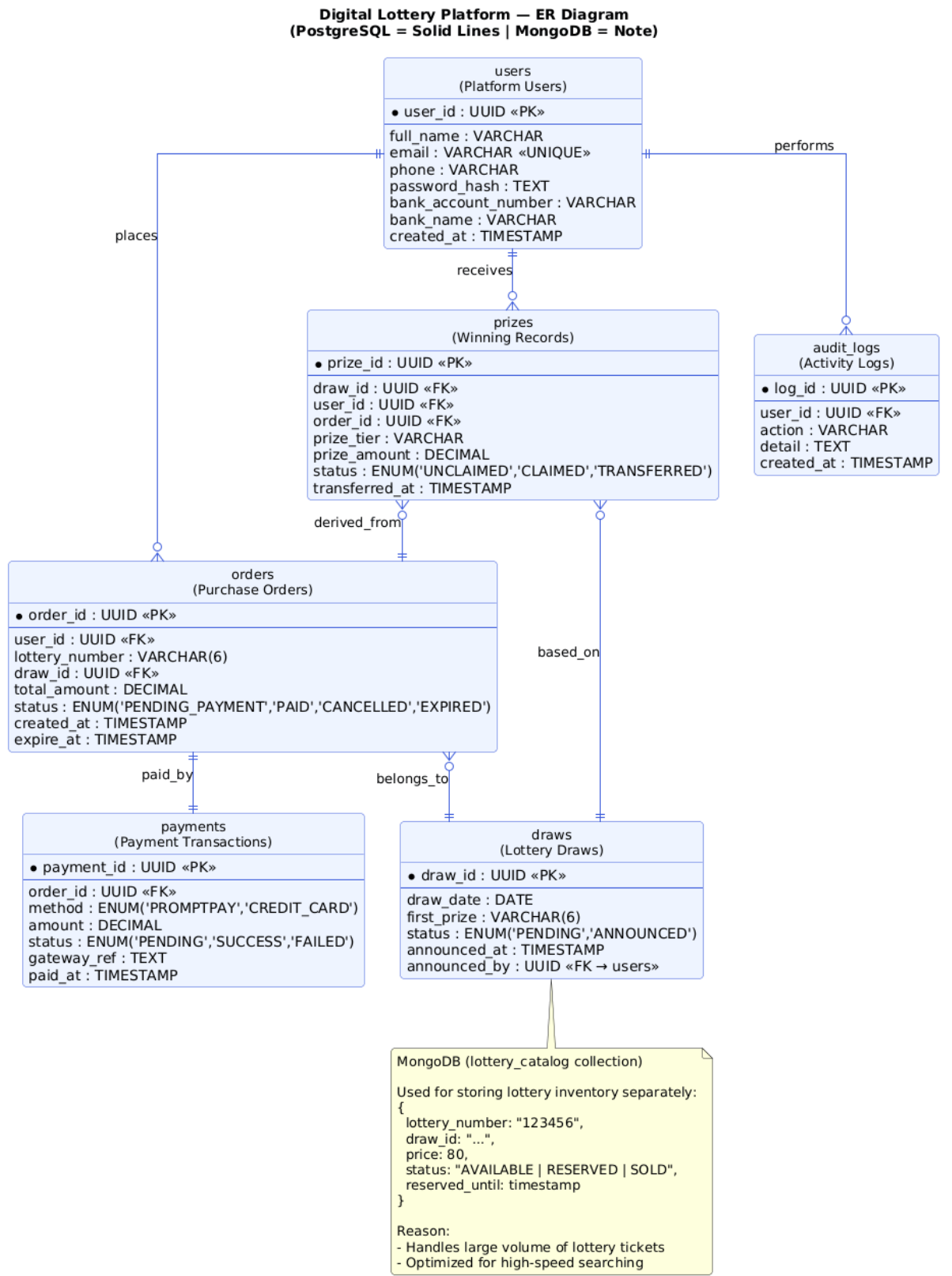
เมื่อการชำระเงินสำเร็จ Payment Service จะดำเนินการ 3 อย่างพร้อมกันแบบ Parallel ได้แก่ เผยแพร่ Purchase Event เข้า RabbitMQ เพื่อส่งต่อให้ Notification Service, บันทึก Audit Log การซื้อสำเร็จ และส่งข้อมูลยืนยันกลับสู่ Frontend Frontend จะแสดงหน้าจอ ยืนยันพร้อมหมายเลขสลาก จำนวนเงิน วันที่ซื้อ และเลขยืนยันการซื้อแก่ผู้ใช้

ขั้นตอนที่ 5 — การแจ้งเตือน (FR-10, UC-10)

Notification Service รับ Purchase Event จาก RabbitMQ Queue แล้วบันทึกข้อมูลการแจ้งเตือนใน PostgreSQL พร้อมสถานะ "รอดำเนินการ" จากนั้นเตรียมข้อความและส่ง Push Notification ผ่าน Firebase FCM ไปยังอุปกรณ์ของผู้ใช้ พร้อมอัปเดตสถานะการแจ้งเตือนเป็น "ส่งแล้ว" ผู้ใช้จะได้รับการแจ้งเตือนทั้งบนแอปพลิเคชัน และ Mobile Push Notification รวมถึงสามารถกำหนดค่าให้รับผ่าน Email ได้ด้วย

NFR-R04: Queue Buffer — หาก Notification Service ล่มชั่วคราว Message จะถูกเก็บรอใน RabbitMQ Queue ไม่สูญหาย

Digital Lottery Platform — Entity Relationship Diagram



ER Diagram นี้แสดงโครงสร้างของระบบลอตเตอรี่ออนไลน์ ซึ่งประกอบด้วยเอนทิตีหลัก ได้แก่ **Users, Draws, Orders, Payments, Prizes** และ **Audit Logs** โดย **Users** เป็นศูนย์กลางของระบบที่เชื่อมโยงไปยังการสั่งซื้อ (**Orders**) การรับรางวัล (**Prizes**) และบันทึกการใช้งาน (**Audit Logs**) ในขณะที่ **Orders** จะเชื่อมกับ **Draws** เพื่อระบุงวดของสลาก และมี **Payments** แบบหนึ่งต่อหนึ่งสำหรับการชำระเงิน

เมื่อมีการประกาศผลรางวัลจาก **Draws** หากคำสั่งซื้อใดถูกรางวัล ระบบจะบันทึกข้อมูลใน **Prizes** พร้อมสถานะการรับและการโอนเงิน นอกจากนี้ยังมี **Audit Logs** สำหรับติดตามการกระทำต่าง ๆ ของผู้ใช้งานเพื่อความปลอดภัยและการตรวจสอบย้อนหลัง

ระบบนี้ใช้ฐานข้อมูลแบบผสม โดย **PostgreSQL** ใช้สำหรับจัดเก็บข้อมูลเชิงธุรกรรม เช่น ผู้ใช้งาน คำสั่งซื้อ และการชำระเงิน เพื่อให้มีความถูกต้องและความสัมพันธ์ของข้อมูลที่ชัดเจน ส่วน **MongoDB** ใช้สำหรับเก็บข้อมูลสลากจำนวนมากในลักษณะ **collection** เพื่อให้สามารถค้นหาและจัดการข้อมูลได้อย่างรวดเร็วและมีประสิทธิภาพ